WDS Food APP — Documentação Técnica

Versão: 1.1.0
Data: Setembro/2026
Autor: Equipe WDS
Público-alvo: Equipe de desenvolvimento

📋 Sumário

1. Visão Geral
2. Arquitetura
3. Stack Tecnológico
4. Estrutura de Pastas
5. Configuração e Instalação
6. Fluxos de Negócio
7. Integração com Backend
8. Endpoints da API
9. Banco de Dados
10. Recursos Configuráveis
11. Proteção de Concorrência
12. Autenticação e Sessão
13. Temas Visuais
14. Build e Publicação
15. Roadmap

---

1. Visão Geral

1.1 Objetivo

O **WDS Food APP** é uma aplicação mobile de controle de mesas para restaurantes, integrada ao ERP legado em VB6 e a uma API PHP intermediária.

1.2 Escopo

- Controle de mesas em tempo real
- Lançamento de pedidos por garçons
- Histórico de vendas

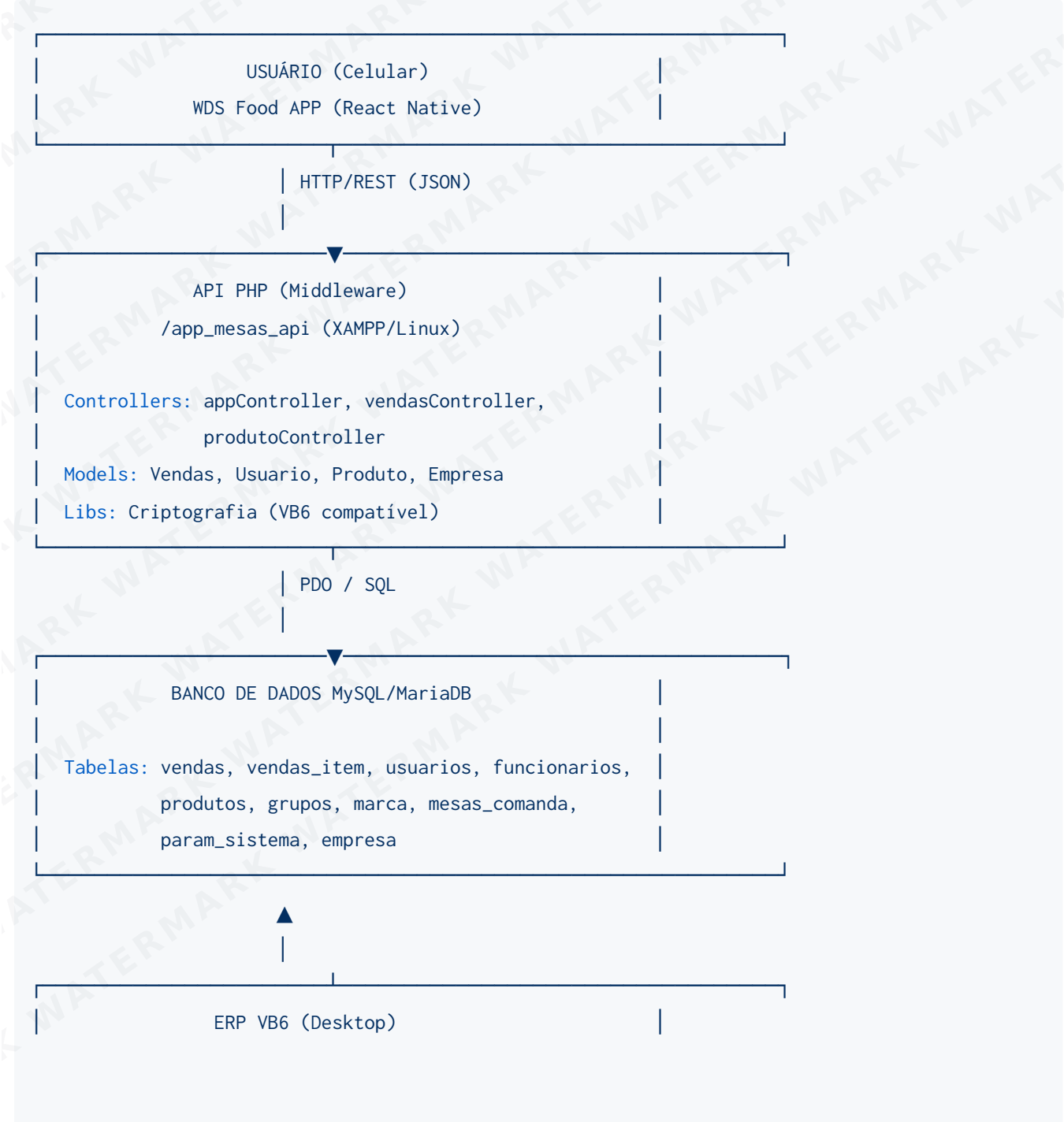
- Configuração multi-empresa e multi-ramo
- Interface adaptável por tipo de estabelecimento

1.3 Público-Alvo

- **Garçons**: lançamento de pedidos
- **Supervisores**: acompanhamento e relatórios
- **Donos/Gerentes**: análise de vendas

---

2. Arquitetura



```
|          Controle de Mesas / PDV / Caixa                    |
└─────────────────────────────────────────────────────────────┘
```

2.1 Componentes

Componente	Tecnologia	Responsabilidade
App Mobile	React Native + Expo SDK 57	Interface com o usuário
API PHP	PHP 7.4+ / 8.x	Camada intermediária REST
Banco de Dados	MySQL 5.7+ / MariaDB	Persistência de dados
ERP	VB6 Desktop	Sistema legado de gestão

---

3. Stack Tecnológico

3.1 Frontend (App Mobile)

Tecnologia	Versão	Uso
Node.js	20.19.5+	Ambiente de build
React Native	0.86.x	Framework mobile
Expo	SDK 57	Toolchain
React Navigation	6.x	Navegação entre telas
Axios	1.20+	HTTP client
AsyncStorage	2.2+	Persistência local

3.2 Backend (API)

Tecnologia	Versão	Uso
PHP	7.4+	Linguagem backend

Tecnologia	Versão	Uso
PDO	-	Acesso ao banco
MySQL	5.7+	Banco de dados
Apache	2.4+	Servidor web

3.3 Build

Ferramenta	Uso
EAS Build	Build na nuvem (APK)
Git	Controle de versão
expo-doctor	Validação de dependências

---

4. Estrutura de Pastas

```
MeuAppMesasV3/
├── App.js                      # Entry point
├── index.js                    # Registro do app
├── app.json                    # Configuração do Expo
├── eas.json                    # Configuração do build
├── package.json                # Dependências
│
├── assets/                     # Recursos estáticos
│   ├── icon.png                # Ícone principal
│   ├── adaptive-icon.png       # Ícone Android
│   ├── splash.png              # Tela inicial
│   └── favicon.png             # Ícone web
│
└── src/
    ├── config/
    │   ├── index.js            # Config de API e endpoints
    │   ├── colors.js           # Paleta de cores
    │   └── themes.js           # Temas por ramo
    │
```

```
├── context/
│   └── AuthContext.js          # Estado global de autenticação
│
├── hooks/
│   ├── useTema.js              # Hook do tema atual
│   ├── useRecursos.js          # Hook das feature flags
│   └── useAutoRefresh.js       # Hook de auto-refresh
│
├── navigation/
│   └── AppNavigator.js         # Configuração de navegação
│
├── services/
│   ├── api.js                  # Axios + interceptors
│   ├── authService.js          # Login/logout
│   ├── configService.js        # Config persistida
│   ├── sessionService.js       # Sessão com expiração
│   ├── mesaService.js          # Mesas
│   ├── pedidoService.js        # Pedidos/itens
│   ├── produtoService.js       # Produtos
│   └── historicoService.js     # Histórico
│
├── screens/
│   ├── ConfigScreen.js         # Configuração inicial/edição
│   ├── LoginScreen.js          # Login
│   ├── DashboardScreen.js      # Painel inicial
│   ├── MesasScreen.js          # Grade de mesas
│   ├── PedidoScreen.js         # Pedido da mesa
│   ├── ProdutosScreen.js       # Cardápio
│   ├── HistoricoScreen.js      # Histórico de vendas
│   └── DetalhesPedidoScreen.js # Detalhes de venda
│
└── utils/
    ├── constants.js            # Constantes
    └── formatters.js           # Formatações
```

5. Configuração e Instalação

5.1 Requisitos do Ambiente de Desenvolvimento

```
# Verificar versões
node --version       # >= 20.19.5
```

```
npm --version        # >= 10.x
nvm --version        # opcional, recomendado
git --version
```

5.2 Clonar e Instalar

```
# Clone
git clone <repositorio>
cd MeuAppMesasV3

# Instalar dependências
npm install

# Validar ambiente
npx expo-doctor
```

5.3 Rodar em Desenvolvimento

```
# Limpar cache e iniciar
npx expo start -c

# Ou para Android específico
npx expo start --android
```

5.4 Configurar a API no Primeiro Acesso

Na primeira execução, o app solicita:

Campo	Exemplo	Descrição
IP/Domínio	`192.168.100.67`	IP do servidor
Porta	`80`	Porta HTTP
Caminho da API	`app_mesas_api`	Pasta da API
Empresa	`1`	Código da empresa
Ramo	Restaurante	Tipo do estabelecimento

O app testa a conexão antes de salvar.

---

6. Fluxos de Negócio

6.1 Fluxo Principal

```
Login → Dashboard → Mesas → Abrir Mesa → Pedido → Adicionar Itens → Fechar (PDV)
                                                          ↓
                                                   Cozinha (ERP)
```

6.2 Fluxo Detalhado

6.2.1 Autenticação

1. Usuário informa login e senha
2. App criptografa a senha (mesma criptografia do VB6)
3. Backend valida com `usuarios.senha`
4. Retorna token + dados (usuário + funcionário + empresa)
5. App salva sessão com validade de 7 dias

6.2.2 Abertura de Mesa

1. Garçom toca em mesa **livre** na grade
2. App confirma abertura
3. Backend cria venda sequencial (`MAX(venda) + 1`)
4. Venda é vinculada à mesa e ao **funcionário** que abriu
5. App navega para a tela de Pedido

6.2.3 Lançamento de Itens

1. Garçom toca em "Adicionar"
2. App mostra cardápio com filtros por grupo
3. Garçom escolhe produto + quantidade + observação
4. Backend valida com **transação**:
   - Se venda está fechada → rejeita (erro 409)
   - Se venda está aberta → adiciona item
5. App volta automaticamente ao Pedido

6.2.4 Fechamento (no PDV)

1. Caixa finaliza venda no PDV
2. Status da venda muda para `1` (fechada)
3. App detecta ao tentar editar → mostra tela de bloqueio
4. PDV emite NFC-e (obrigatório em MT)

7. Integração com Backend

7.1 Formato de Comunicação

- **Protocolo:** HTTP/1.1
- **Formato:** JSON
- **Método:** POST (na maioria dos endpoints)
- **Autenticação:** Bearer Token (no header `Authorization`)

7.2 Estrutura de Configuração

Arquivo: `src/config/index.js`

```
export const API_CONFIG = {
  baseURL: '',  // Preenchido dinamicamente
  timeout: 30000,
  endpoints: {
    login: '/app/login',
    mesasListar: '/vendas/listar',
    vendaAbrir: '/app/vendaAbrir',
    itemAdicionar: '/app/itemAdicionarSeguro',
    itemListar: '/vendas/itemlistar',
    itemCancelar: '/vendas/itemcancelar',
    produtosListar: '/produto/listarConsulta',
    gruposListar: '/produto/listarGrupos',
    historicoListar: '/app/historicoListar',
    historicoDetalhes: '/app/historicoDetalhes',
    verificarStatus: '/app/verificarStatus',
  }
};
```

7.3 Interceptors do Axios

Arquivo: `src/services/api.js`

O axios possui 2 interceptors:

1. **Request interceptor:**
    - Carrega `baseURL` do AsyncStorage
    - Adiciona token de autenticação
2. **Response interceptor:**
    - Renova sessão se próximo de expirar

- Limpa HTML de erros do PHP
- Trata erro 401 (sessão inválida)

---

8. Endpoints da API

8.1 Autenticação

Endpoint	Método	Body	Retorno
`/app/login`	POST	`{usuario, senha, empresa}`	`{sucesso, token, usuario}`
`/app/ping`	GET	-	`{status: "ok"}`

8.2 Mesas

Endpoint	Método	Body	Retorno
`/vendas/listar`	POST	`{empresa}`	Array de mesas
`/app/vendaAbrir`	POST	`{empresa, mesa, vendedor}`	`{venda, ja_aberta}`
`/app/verificarStatus`	POST	`{empresa, venda}`	`{aberta, fechada}`

8.3 Pedidos

Endpoint	Método	Body	Retorno
`/app/itemAdicionarSeguro`	POST	`{empresa, venda, referencia, ...}`	`{sucesso, item}`
`/vendas/itemlistar`	POST	`{empresa, venda}`	Array de itens
`/vendas/itemcancelar`	POST	`{empresa, venda, item}`	`{sucesso}`

8.4 Produtos

Endpoint	Método	Body	Retorno
`/produto/listarConsulta`	POST	`{empresa, texto}`	Array de produtos

Endpoint	Método	Body	Retorno
`/produto/listarGrupos`	POST	`{empresa}`	Array de grupos

8.5 Histórico

Endpoint	Método	Body	Retorno
`/app/historicoListar`	POST	`{empresa, periodo, status}`	Array de vendas
`/app/historicoDetalhes`	POST	`{empresa, venda}`	`{cabecalho, itens}`

---

9. Banco de Dados

9.1 Tabelas Principais

`vendas`

Coluna	Tipo	Descrição
empresa	INT	Código da empresa
venda	INT	Código sequencial da venda (PK)
mesa_comanda	INT	Mesa vinculada
vendedor	INT	**Código do FUNCIONÁRIO**
status	INT	0=aberta, 1=fechada, 2=cancelada
total	DECIMAL	Total da venda
forma_pgto	INT	Forma de pagamento
origem	VARCHAR	APP, MESAS, PDV, ORC
data	DATE	Data da venda
hora_venda	TIME	Hora da venda

`vendas_item`

Coluna	Tipo	Descrição
empresa	INT	Código da empresa
venda	INT	Venda (FK)
item	INT	Número do item
referencia	VARCHAR	Código do produto
quantidade	DECIMAL	Quantidade
preco	DECIMAL	Preço unitário
total_liquido	DECIMAL	Total do item
cancelado	INT	0=ativo, 1=cancelado
observacao	TEXT	Observação do item

`usuarios`

Coluna	Tipo	Descrição
codigo	INT	Código (PK)
empresa	INT	Código da empresa
nome	VARCHAR	Nome de login
senha	VARCHAR	Senha criptografada (VB6)
status	INT	0=ativo

`funcionarios`

Coluna	Tipo	Descrição
codigo	INT	Código (PK)

Coluna	Tipo	Descrição
empresa	INT	Código da empresa
nome	VARCHAR	Nome do funcionário
USER	INT	Código do usuário vinculado

⚠️ **Importante:** `funcionarios.USER` é palavra reservada. Use `` `USER` `` (com backticks) em queries SQL.

9.2 Relação Usuário ↔ Funcionário

```
usuarios.codigo ──1:1──► funcionarios.USER
     (login)                (vendedor nas vendas)
```

Ao gravar em `vendas.vendedor`, sempre usar o código do FUNCIONÁRIO, não o do usuário.

---

10. Recursos Configuráveis

10.1 Feature Flags

Arquivo: `src/services/configService.js`

```
export const RECURSOS_PADRAO = {
  finalizarVenda: false,       // Fechar pedido pelo app
  visualizarRelatorios: true,  // Tela de relatórios
  visualizarHistorico: true,   // Histórico de pedidos
};
```

10.2 Como Adicionar Novo Recurso

1. Adicione a chave em `RECURSOS_PADRAO`
2. Adicione um switch em `ConfigScreen.js`
3. Use `useRecursos()` nas telas que precisam:

```
const recursos = useRecursos();
{recursos.novoRecurso && <Componente />}
```

---

11. Proteção de Concorrência

11.1 Cenários de Conflito

Cenário	Solução
2 garçons adicionam item na mesma mesa	Transação SQL (FOR UPDATE)
PDV fecha mesa enquanto app edita	Verificação + tela de bloqueio
2 apps fecham a mesma mesa	UPDATE atômico (status = 0)

11.2 Implementação

Adicionar Item (transação)

```
$this->db->beginTransaction();
$lock = $this->db->prepare("SELECT venda, status FROM vendas
    WHERE empresa = :empresa AND venda = :venda FOR UPDATE");
// ... se status != 0 → rollback + 409
// ... se ok → adiciona item
$this->db->commit();
```

Detecção no App

```
// PedidoScreen.js
useFocusEffect(useCallback(() => {
  const status = await mesaService.verificarStatus(venda);
  if (status.fechada) setVendaFechada(true);
}, [venda]));
```

Tela de Bloqueio

Quando `vendaFechada = true`, renderiza tela dedicada com botão "Voltar para Mesas" que faz `navigation.reset()`.

---

12. Autenticação e Sessão

12.1 Criptografia de Senha

O app usa a **mesma criptografia do ERP VB6**:

Algoritmo (traduzido para PHP):

```php
public static function encriptar($texto) {
    $texto = strtolower($texto);
    $resultado = '';
    for ($i = 0; $i < strlen($texto); $i++) {
        $caracter = ord($texto[$i]) * 12 + 100000;
        $resultado .= substr((string)$caracter, 1);
    }
    return $resultado;
}
```

Exemplo: `w31d48s38` → `0142800612005880120000624006720138000612006 72`

12.2 Sessão com Expiração

Arquivo: `src/services/sessionService.js`

- **Duração:** 7 dias
- **Renovação:** automática em cada requisição (se faltar < 3 dias)
- **Armazenamento:** AsyncStorage (`@app:token`, `@app:user`, `@app:expiresAt`)

12.3 Estrutura do Token

Token simples em base64:

```
base64_encode(codigo . ':' . time() . ':' . md5(codigo . 'chave_secreta'))
```

Não é JWT, mas cumpre a função de validar a sessão.

---

13. Temas Visuais

13.1 Ramos Disponíveis

Arquivo: `src/config/themes.js`

ID	Nome	Ícone	Cor
restaurante	Restaurante	🍽️	#3498db (azul)
lanches	Lanches	🍔	#e67e22 (laranja)

ID	Nome	Ícone	Cor
sorveteria	Sorveteria	🍦	#9b59b6 (roxo)
espetinhos	Espetinhos	🍢	#e74c3c (vermelho)
bar	Bar	🍺	#8e44ad (roxo escuro)

13.2 Como Usar

```
import { useTema } from '../hooks/useTema';

const tema = useTema();
// Acesse: tema.icone, tema.cor, tema.icones.mesas, etc
```

13.3 Como Adicionar Novo Ramo

1. Adicione uma entrada em `TEMAS` (`src/config/themes.js`)
2. Pronto! O app já mostra automaticamente na tela de config

---

14. Build e Publicação

14.1 Build do APK (EAS)

```
# Configurar (primeira vez)
npx eas-cli build:configure

# Build de preview (APK interno)
npx eas-cli build --platform android --profile preview

# Build de produção (para publicar)
npx eas-cli build --platform android --profile production
```

14.2 Requisitos para o `app.json`

```
{
  "expo": {
    "name": "WDS Food",
    "slug": "wds-food-app",
    "android": {
      "package": "com.wds_comanda.app",
      "versionCode": 1
```

```
    },
    "plugins": [
      ["expo-build-properties", {
        "android": { "usesCleartextTraffic": true }
      }]
    ]
  }
}
```

⚠️ **Importante:** `usesCleartextTraffic` deve estar **dentro do plugin** `expo-build-properties` (não diretamente no `android`) no SDK 57.

14.3 APK de Preview

Após o build, o link é fornecido pelo EAS:

```
https://expo.dev/accounts/<usuario>/projects/wds-food-app/builds/<id>
```

Baixe, envie para o celular, instale e teste.

---

15. Roadmap

15.1 Implementado (v1.1)

- ✅ Login com sessão persistente (7 dias)
- ✅ Dashboard com estatísticas
- ✅ Mesas com auto-refresh (10s)
- ✅ Abertura de mesa vinculada ao funcionário
- ✅ Pedido com adicionar/remover itens
- ✅ Produtos com filtro por grupo
- ✅ Configurações multi-empresa
- ✅ 5 temas visuais
- ✅ 3 feature flags
- ✅ Proteção de concorrência
- ✅ Histórico de pedidos
- ✅ Nome do funcionário em toda a UI

15.2 Próximos Passos

Curto Prazo

- 🔶 Tela de Relatórios (vendas por período, produtos top)

- 🔶 Multi-empresa dinâmico (trocar empresa sem trocar IP)
- 🔶 Setores do salão (agrupar mesas por área)
- 🔶 Fechamento pelo app (Fase 1 - sem NFC-e)

Médio Prazo

- 🔶 Integração com Smart POS parceiro
- 🔶 Totem de autoatendimento
- 🔶 Notificações push
- 🔶 Modo offline

Longo Prazo

- 🔶 Emissão de NFC-e (própria)
- 🔶 Programa de fidelidade
- 🔶 Cardápio digital com QR Code

📝 Anexos

A. Comandos Úteis

```
# Desenvolvimento
npx expo start -c              # Iniciar com cache limpo
npx expo-doctor                # Validar dependências
npx expo export --platform android  # Testar bundle

# Git
git add .
git commit -m "mensagem"
git tag vX.Y.Z

# EAS
npx eas-cli build --platform android --profile preview
npx eas-cli build:list
```

B. Variáveis de Ambiente no `app.json`

```
{
  "extra": {
    "eas": {
      "projectId": "<UUID do projeto EAS>"
    }
```

```
  }
}
```

C. Estrutura de Pastas do Backend

```
app_mesas_api/
├── index.php
├── core/
│   └── Core.php
├── controllers/
│   ├── appController.php
│   ├── vendasController.php
│   └── produtoController.php
├── models/
│   ├── Vendas.php
│   ├── Usuario.php
│   ├── Produto.php
│   └── Empresa.php
└── lib/
    └── Criptografia.php
```

Fim do documento

Última atualização: Setembro/2026

WATERMARK WATERMARK WATERMARK